PRAKTIKUM TEKNOLOGI IOT II

LAPORAN PRAKTIK

TEKNOLOGI IOT II

Disusun dalam rangka memenuhi salah satu syarat kelengkapan nilai Praktikum Mata Kuliah Teknologi IOT II

DISUSUN OLEH:

NAMA : Tengku Geisya Gustiana Syah

NPM : 25782028

KELAS :3A

PRODI : TEKNOLOGI REKAYASA INTERNET

Dosen :

Dr. Ir. Septafiansyah Dwi Putra, S. T., M.T.

Agus Ambarwari, S.Pd.,M.kom,

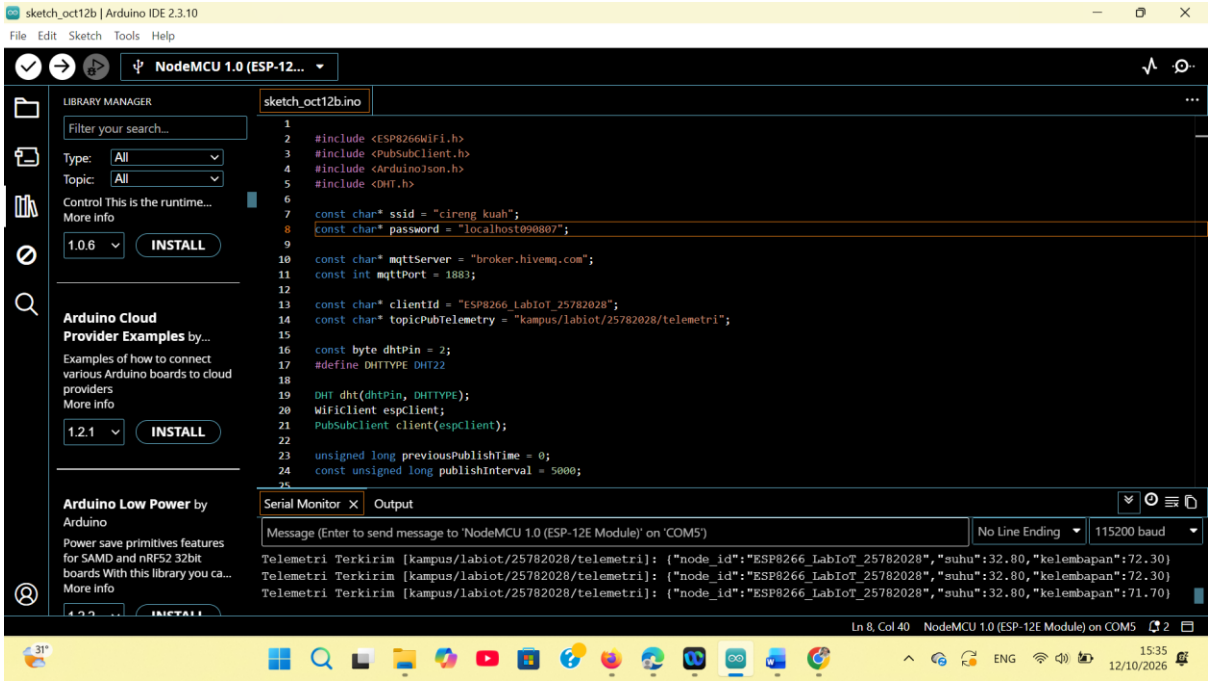
TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG

2026

6..5.1 Praktikum 1: Publikasi Data Sensor Lingkungan (DHT) Berformat JSON

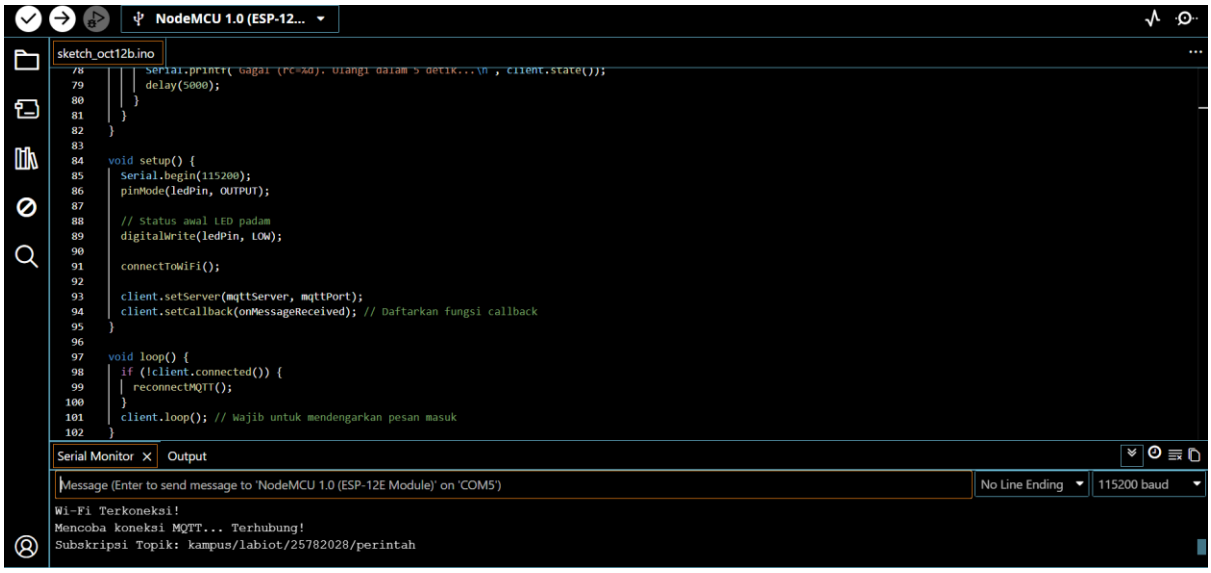


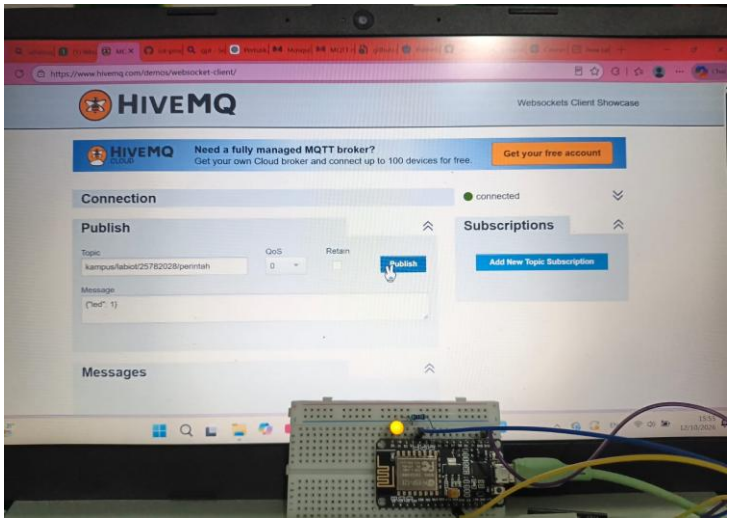
Pada praktikum ini dilakukan pengiriman data telemetri dari ESP8266 menggunakan protokol MQTT. ESP8266 dihubungkan ke jaringan Wi-Fi, kemudian dikonfigurasi agar dapat terhubung dengan broker MQTT. Sensor DHT digunakan untuk membaca data suhu dan kelembapan lingkungan.

Data yang diperoleh dari sensor disusun dalam format JSON menggunakan library ArduinoJson. Data tersebut kemudian dikirimkan secara berkala setiap lima detik ke topik MQTT yang telah ditentukan. Penggunaan format JSON mempermudah penyusunan dan pembacaan data telemetri sebelum dikirimkan ke broker.

Berdasarkan pengujian, koneksi Wi-Fi perlu berhasil terlebih dahulu sebelum proses pengiriman data MQTT dapat dilakukan. Pada pengujian yang dilakukan, ESP8266 belum berhasil terhubung ke Wi-Fi sehingga pengiriman data telemetri ke broker MQTT belum dapat diverifikasi. Oleh karena itu, koneksi Wi-Fi perlu diperiksa sebelum pengujian dilanjutkan.

6.5.2 Praktikum 2: Subskripsi dan Kontrol Indikator LED Asinkron







Pada praktikum ini dilakukan pengujian kendali LED menggunakan protokol MQTT. ESP8266 dikonfigurasi sebagai subscriber untuk menerima pesan dari broker MQTT melalui topik kendali yang telah ditentukan. Pesan yang diterima digunakan sebagai perintah untuk mengatur kondisi LED, seperti menyalakan atau mematikan LED.

Program ESP8266 harus tetap memproses pesan MQTT secara berkala agar perintah yang dikirimkan dapat diterima tanpa menghambat proses lainnya. Dengan mekanisme tersebut, LED dapat dikendalikan melalui pesan yang dipublikasikan ke topik MQTT.

Keberhasilan praktikum ditandai dengan ESP8266 yang berhasil terhubung ke Wi-Fi dan broker MQTT, menerima pesan kendali, serta mengubah kondisi LED sesuai perintah. Namun, pengujian kendali LED belum dapat diverifikasi apabila koneksi Wi-Fi dan MQTT belum berhasil dilakukan.

6.5.3 Praktikum 3: Sistem Terpadu Dua Arah (Full Duplex Telemetry & Control)



Pada praktikum ini dilakukan penggabungan fungsi pengiriman data telemetri dan penerimaan perintah kendali dalam satu perangkat ESP8266. Perangkat membaca data suhu dan kelembapan dari sensor DHT, kemudian mengirimkannya ke broker MQTT dalam format JSON. Selain itu, ESP8266 menerima pesan kendali dari topik MQTT untuk mengatur kondisi LED.

Komunikasi dua arah ini memungkinkan perangkat mengirimkan informasi kondisi lingkungan sekaligus menerima perintah dari pengguna. Data telemetri berfungsi sebagai informasi pemantauan, sedangkan pesan kendali digunakan untuk mengatur keluaran perangkat.

Keberhasilan praktikum ditandai dengan data sensor yang dapat diterima melalui topik telemetri dan perintah kendali yang dapat mengubah kondisi LED. Kedua fungsi tersebut memerlukan koneksi Wi-Fi

dan koneksi broker MQTT yang stabil. Karena koneksi Wi-Fi pada pengujian belum berhasil, komunikasi dua arah belum dapat dipastikan berjalan sesuai tujuan praktikum.

6.5.4 Panduan Pengujian Diagnostik dengan MQTT Explorer

Pada praktikum ini digunakan aplikasi MQTT Explorer untuk memantau komunikasi antara perangkat ESP8266 dan broker MQTT. Aplikasi tersebut membantu pengguna melihat topik MQTT, pesan yang dipublikasikan, serta data yang diterima oleh broker.

Pengujian dilakukan dengan menghubungkan MQTT Explorer ke broker yang digunakan oleh ESP8266. Setelah koneksi berhasil, pengguna dapat memantau topik telemetri untuk melihat data sensor dalam format JSON serta mengirim pesan ke topik kendali untuk menguji fungsi subscriber pada ESP8266.

MQTT Explorer juga dapat digunakan untuk membantu proses diagnostik apabila pesan tidak diterima atau tidak muncul pada topik yang diharapkan. Pemeriksaan dapat dilakukan terhadap alamat broker, port, nama topik, dan status koneksi perangkat.

Keberhasilan praktikum ditandai dengan MQTT Explorer dapat terhubung ke broker dan menampilkan pesan pada topik yang sesuai. Namun, karena ESP8266 belum berhasil terhubung ke Wi-Fi pada pengujian sebelumnya, pesan telemetri dari perangkat belum dapat diverifikasi melalui MQTT Explorer.

6.6 Latihan

Latihan 1. Analisis Arsitektur dan Perbandingan Protokol

1. Efisiensi penggunaan bandwidth pada paket data seluler

MQTT lebih efisien dibandingkan HTTP karena memiliki ukuran header yang lebih kecil. MQTT menggunakan model Publish-Subscribe sehingga perangkat cukup mengirim pesan ke broker tanpa harus membuat permintaan HTTP berulang kali kepada server. Hal ini dapat mengurangi penggunaan bandwidth, terutama pada jaringan seluler yang memiliki kapasitas terbatas. Sebaliknya, HTTP memiliki overhead permintaan dan respons yang relatif lebih besar.

2. Beban komputasi mikrokontroler (CPU overhead) saat menangani pertukaran data dua arah

MQTT memiliki beban komputasi yang relatif ringan karena format paketnya sederhana dan komunikasi dikelola melalui broker. Perangkat dapat mengirim data sensor sekaligus menerima perintah melalui mekanisme Publish-Subscribe. Pada HTTP, perangkat harus menangani proses permintaan dan respons, termasuk header yang lebih besar. Oleh karena itu, MQTT lebih sesuai untuk mikrokontroler dengan sumber daya komputasi dan memori terbatas.

3. Kemampuan pengiriman data secara simultan ke banyak perangkat penerima (1-to-N distribution)

MQTT mendukung pengiriman data ke banyak penerima melalui broker. Publisher cukup menerbitkan pesan pada satu topik, kemudian broker meneruskan pesan tersebut kepada semua subscriber yang berlangganan pada topik yang sesuai. Dengan mekanisme ini, publisher tidak perlu mengirim pesan secara terpisah kepada setiap perangkat penerima. Hal tersebut membuat komunikasi lebih efisien dan mudah dikembangkan ketika jumlah perangkat bertambah.

Latihan 2. Analisis Hierarki Topik dan Karakter Wildcard

Struktur topik yang digunakan adalah pabrik/<line>/<mesin>/<sensor>.

1. Memantau parameter suhu dari seluruh mesin di semua lini produksi

Topik subskripsi:

pabrik/+/+/suhu

Karakter + menggantikan tepat satu tingkat hierarki. Wildcard pertama mencocokkan semua lini produksi, sedangkan wildcard kedua mencocokkan semua mesin. Dengan demikian, subscriber menerima data suhu dari seluruh mesin di semua lini.

2. Memantau seluruh parameter sensor khusus untuk mesin m1 di lini line2

Topik subskripsi:

pabrik/line2/m1/+

Wildcard + pada tingkat terakhir mencocokkan semua parameter sensor pada mesin m1 di line2, termasuk suhu, getaran, dan status.

3. Memantau seluruh lalu lintas data dari seluruh lini dan mesin di pabrik

Topik subskripsi:

pabrik/#

Karakter # menggantikan seluruh tingkat hierarki setelah awalan pabrik/. Dengan demikian, subscriber dapat menerima semua pesan pada topik di bawah hierarki pabrik tersebut.

Latihan 3. Implementasi Serialisasi Data Dinamis

Untuk menambahkan informasi lama waktu aktif dan alamat IP lokal ke paket telemetri JSON, tambahkan baris berikut pada bagian pembentukan objek JSON:

```
doc["uptime_detik"] = millis() / 1000;

doc["ip_lokal"] = WiFi.localIP().toString();
```

Keterangan:

1. uptime_detik menyimpan lama waktu aktif perangkat dalam satuan detik. Nilainya diperoleh dari millis() yang dibagi 1000.
2. ip_lokal menyimpan alamat IP lokal ESP8266 dalam bentuk string melalui WiFi.localIP().toString().

Kedua parameter tersebut membuat paket telemetri memberikan informasi tambahan mengenai waktu aktif perangkat dan alamat IP yang digunakan.

Latihan 4. Analisis Mekanisme Reconnect dan Loop Blocking

1. Konsekuensi penggunaan delay(6000) terhadap responsivitas callback

Pemanggilan delay(6000) membuat program berhenti menjalankan proses berikutnya selama enam detik. Jika instruksi tersebut ditempatkan di dalam fungsi loop(), pemanggilan client.loop() juga menjadi terlambat. Akibatnya, pesan MQTT yang masuk tidak segera diproses dan respons terhadap perintah kendali LED menjadi lambat. Kondisi ini mengurangi responsivitas komunikasi asinkron.

2. Cara kerja fungsi reconnect() dalam memulihkan koneksi

Fungsi reconnect() digunakan untuk mencoba membangun kembali koneksi MQTT ketika koneksi ke broker terputus. Program memeriksa status koneksi, kemudian mencoba menghubungkan kembali ESP8266 ke broker menggunakan konfigurasi yang telah ditentukan. Setelah koneksi berhasil, perangkat dapat kembali melakukan publikasi data telemetri dan menerima pesan dari topik yang telah disubskripsikan. Jika koneksi Wi-Fi terputus, koneksi Wi-Fi perlu dipulihkan terlebih dahulu sebelum koneksi MQTT dapat dibangun kembali.